

오늘도 장비 대여

Today's Weapon Rental — 모험가에게 무기를 빌려주고 던전을 주선하는 길드 경영 시뮬레이션

윤승희 게임 클라이언트 프로그래머 지원 · 부경대학교 졸업 예정

dldudcjs12345@gmail.com

Google Play play.google.com/store/apps/details?id=com.IdeaBank.TodaysWeaponRental

GitHub github.com/seunghee-2002/Todays-Weapon-Rental-code

소개 페이지 seunghee-2002.github.io/todays-weapon-rental



CONTENTS

목차

1. 소개	3
2. 프로젝트 개요 — 오늘도 장비 대여	4
게임 소개 · 하루의 흐름 · 주요 시스템 · 프로젝트 구조	
3. 개발 내용	6
3-1. 모험 시스템 — 최대 성공률이 70%인 문제	6
3-2. 저장 시스템 — 반복되던 데이터 손실	8
3-3. 온라인 — 리더보드 치팅과 세이브 동기화	9
3-4. 밸런싱 — 근거 없는 수치 조정	11
3-5. 다국어 — 3개 언어 추가	13
3-6. 기획 데이터 툴 — 조용히 깨지는 데이터	14
4. 트러블슈팅	15
5. 출시와 운영	16
6. 회고	17

1

소개

혼자서 기획부터 출시까지 끝까지 가 본 경험이 있습니다. 문제가 생기면 임시로 막기보다 원인을 측정하고, 같은 문제가 다시 나오지 않게 장치를 남기는 방식으로 일합니다.

프로필

이름	윤승희
지원 직군	게임 클라이언트 프로그래머
학력	부경대학교 졸업 예정
연락처	dldudcjs12345@gmail.com
GitHub	github.com/seunghee-2002/Todays-Weapon-Rental-code

기술 스택

주력	Unity 2022.3 · C# — uGUI, TextMeshPro, ScriptableObject 기반 데이터, Addressables, Spine 런타임, 에디터 확장(EditorWindow) 제작
온라인	Unity Gaming Services — Authentication, Cloud Save, Cloud Code(JavaScript), Leaderboards, Analytics
보조	Python(FastAPI, 운영 툴) · JavaScript(Cloud Code 서버 함수) · PowerShell(검사 스크립트) · GitHub Actions · Git

프로젝트

오늘도 장비 대여	길드 경영 시뮬레이션 · Android · 1인 개발 · 2025.12 – 2026.08 기획 · 클라이언트 · 서버 함수 · 운영 툴 · 밸런싱 · 스토어 출시를 혼자 진행했습니다. 알파 테스트 3회를 거쳐 2026년 8월 Google Play에 정식 출시했고 현재 운영 중입니다. 이 포트폴리오는 이 프로젝트 하나를 깊이 다룹니다.
-----------	---

프로젝트 개요 — 오늘도 장비 대여

플레이어는 작은 마을의 길드 매니저입니다. 찾아오는 모험가에게 오늘 열린 던전을 골라 주고 무기를 빌려주면, 모험가는 던전을 다녀와 수수료를 냅니다. 성공률을 올리려면 모험가의 숨은 정보를 사야 하고, 그 비용이 수익으로 돌아오는지가 이 게임의 핵심 판단입니다.

기본 정보

장르	길드 경영 시뮬레이션 (로그라이트 회차 구조)
플랫폼	Android — Google Play 정식 출시 v1.0.0 (2026.08)
엔진	Unity 2022.3 LTS · C#
기간 · 인원	2025.12 – 2026.08 · 1인
담당	기획 · 클라이언트 · 서버 함수 · 밸런싱 · 운영 툴 · 출시

하루의 흐름

실시간 1초가 게임 3분입니다. 하루는 네 구간으로 흐르고, 밤이 되면 자동으로 멈춰 다음 날로 넘어갑니다. 주말마다 주간 의뢰를 판정해 미달이면 벌금이 나가고, 골드가 바닥나면 회차가 끝납니다. 끝난 회차는 **유산**(영구 업그레이드)을 남겨 다음 회차가 조금 쉬워집니다.

아침 AM 6 – 9

이벤트 NPC
응대 (대장장이·무기상·상자), AM 9 의뢰판 갱신

낮 AM 9 – PM 6

모험가 방문
→ 정보 공개 (점술·수색·대화) → 던전·무기 배정

저녁 PM 6 – 9

전령 보고 — 모험 결과 정산, 재료 회수

밤 PM 9

자동 일시정지 → 다음 날. 주말엔 주간 의뢰 판정



모험가 방문



점술로 정보 공개



모험 실시간 진행



정산 — 수수료 구조

주요 시스템

시스템	내용	포트폴리오에서
모험	모험가·무기·던전 조합으로 성공률을 계산하고, 이벤트 시퀀스(입장 → 전투·보물·함정 ... → 보스)를 인게임 시간에 맞춰 자동 진행. 결과는 전령 보고 후 플레이어가 확인해야 지급	3-1
모험가 · 정보 공개	방문 모험가의 스탯·특성·던전 방어구 타입은 가려져 있음. 점술(운세)·수색(스탯 공개)·대화(호감도)로 정보를 사서 판단 근거를 확보	2장
무기 · 대장간	무기 타입 × 던전 방어구 상성, 부가효과 5종. 강화·진화·재부여·분해·제작	3-4
의뢰 · 평판	주간 의뢰(40주 캠페인 + 난이도 등급이 붙은 엔드리스), 평판 등급이 모험가 방문 간격을 결정	3-4
유산	회차 종료 시 획득, 영구 업그레이드 20종	3-4
저장	회차 데이터 / 영구 데이터 이원화, 손상 복구, 클라우드 동기화	3-2, 3-3
온라인	익명 로그인, 리더보드(생존 일수), 닉네임, 기기 이전 코드, 계정 차단, 강제 업데이트	3-3
다국어	한국어 · 영어 · 일본어 · 중국어(간체)	3-5

프로젝트 구조 (요약)

규모가 커질 것을 감안해 처음부터 역할을 나눴습니다. 게임 규칙은 **Manager**(싱글턴, C# 이벤트로 변화를 알림)에만 두고, **View**는 표시만, **Controller**가 둘을 잇습니다. Manager는 View를 참조하지 않습니다. 밸런스 수치는 전부 ScriptableObject(Config)에 있어 코드 수정 없이 조정할 수 있고, 기획 데이터(무기·던전·의뢰 등)는 CSV로 왕복 편집합니다(3-6). 저장은 한 회차의 **GameData**와 계정에 누적되는 **PlayerData**를 분리했습니다.

3 — 개발 내용

3-1. 모험 시스템 — 최대 성공률이 70%인 문제

모험은 이 게임의 핵심 루프입니다. 플레이어의 모든 투자(무기 등급, 정보 구매, 대장간)가 결국 "성공률"이라는 숫자 하나로 모이기 때문에, 이 숫자가 투자에 정직하게 반응하지 않으면 게임 전체가 무의미해집니다.

이 시스템이 하는 일

출발 시 던전 등급에 따라 **이벤트 시퀀스**를 만듭니다(Common 3개 ~ Legendary 10개, 첫 칸 입장·마지막 칸 보스 고정, 중간은 던전 풀에서 가중치 추첨). 인게임 시간이 흐를 때마다 한 칸씩 처리하고, 전투·미니보스·보스는 매번 성공률을 굴립니다. 실패 시 보호 횟수가 있으면 재시도, 없으면 후퇴, 보스 실패는 사망 판정까지 갑니다.

성공률은 다음 순서로 계산됩니다.

- | | | |
|----------|---------------------------------------|---------------|
| ① 스탯 점수 | = 모험가 STR/DEX/INT/LUK을 무기 타입별 가중치로 합산 | |
| ② 기본 비율 | = 스탯 점수 ÷ 던전 요구치 | (0~1로 자름) |
| ③ 곱 보정 | = ② × (1 + 무기-방어구 상성 + 부적) | ← 상성 최대 +0.60 |
| ④ 가산 보정 | + 조건부 효과 + 수집 보너스 + 호감도 + 특성 | |
| ⑤ 이벤트 계수 | × 난이도 계수 (보스 0.7 / 미니보스 0.85) | |
| ⑥ 최종 | + 운세 보정, × 기분 배율, 0~1로 자름 | |



모험 진행 — 이벤트 한 칸씩 처리

문제 — 무엇을 해도 보스 성공률은 70%에서 멈췄다

시뮬레이션 결과에서 "완주 확률 상한이 전 등급 약 70%로 같다"는 관측이 있었습니다. 처음엔 등급 간 상대 밸런스만 보고 문제없다고 넘겼는데, 성장 곡선 관점에서 다시 보니 두 가지 결함이 겹쳐 있었습니다.

- **난이도 계수가 최종 성공률에 곱해졌습니다.** ⑤에서 0.7을 곱하니 Legendary 무기 + 모든 스탯 100이라도 보스 성공률은 70%가 상한이었습니다. 그 위를 넘길 수단은 운세(유료 도박)와 기분(랜덤)뿐 — 투자로는 통제할 수 없었습니다.
- **포화 지점이 너무 일렀습니다.** ③이 0~1로 잘리는데 상성 최대가 +0.60이라, 기본 비율이 0.625만 넘으면 1.0으로 잘렸습니다. Common 던전(요구치 110) 기준 스탯 점수 69면 이미 포화 — 그 위의 무기 등급·스탯·도감·호감도 보너스가 전부 사장됐습니다.

즉 "만렙 투자가 70%"가 아니라 "어중간한 투자로 70%에 도달하고 그 이상은 전부 무의미"한 구조였습니다.

해결 — 계수를 "성공률 곱"에서 "요구치 배수"로 옮기고, 중간 클램프를 제거

Config 값(0.7 / 0.85)은 그대로 두고 **해석과 적용 지점만** 바꿨습니다. 난이도 계수 c 를 요구 스탯 임계값의 $1/c$ 배로 해석하면, 보스는 "성공률을 깎는 이벤트"가 아니라 "더 높은 스탯을 요구하는 이벤트"가 됩니다. 투자하면 100%까지 갈 수 있고, 덜 투자하면 그만큼 낮아집니다. 상성·부적 곱 뒤의 클램프도 없애 상위 투자가 결과에 반영되게 했습니다.

```
// AdventureManager.Calculations.cs - CalculateSuccessRate
// 전: effectBase = clamp(스탯점수 / 임계값) ... rate = 최종성공률 × difficulty
// 후: effectBase = clamp(스탯점수 × difficulty / 임계값, 0, 1)
//     baseRate   = effectBase × (1 + 상성 + 부적) // 2차 클램프 제거
public float CalculateSuccessRate(AdventurerInstance adv, WeaponInstance weapon,
                                   DungeonData dungeon, float difficultyMultiplier = 1f)
```

던전 (요구치)	일반 전투	미니보스 (×1.18)	보스 (×1.43)
Common (110)	110	129	157
Rare (180)	180	212	257
Legendary (280)	280	329	400

같은 계산식을 쓰는 밸런스 시뮬레이터(3-4)의 미리 함수도 함께 고쳐 전후를 실측했습니다.

결과 — 투자가 성공률에 반영되고, 초급자 회차 수명도 늘었다

주차별 완주율 (중앙값)	상급자	중급자	초급자	기타	전	후
10주	43% → 60%	53% → 82%	33% → 42%	초급자 회차 수명	30주	44주
21주	50% → 65%	64% → 93%	29% → 39%	상급자 사망률 (판정식은 무수정)	1.4%	0.6%
70주	49% → 69%	67% → 91%	—	모험 1회 순수입 (41~50주)	999G	927G

페르소나 셋 다 완주율이 올랐고 순서는 유지됐습니다. 모험 1회 순수입은 거의 변하지 않아 경제 밸런스는 흔들리지 않았음을 확인했습니다.

플레이테스트 피드백 — "성공률 숫자가 뭘 뜻하는지 모르겠다"

알파 테스트에서 가장 먼저 나온 지적입니다. 위 계산이 6단계나 되니 숫자 하나만 보여 주면 플레이어는 자기 선택이 어디에 영향을 줬는지 알 수 없었습니다. 문구를 고치는 대신 **성공률을 누르면 요소별 기여도(상성·아이템·호감도·이벤트별 확률)를 펼쳐 보여 주는 상세 패널**을 만들었습니다. 표시 색상·임계값은 Config로 빼서 기획 조정이 가능하게 했습니다.

코드: Scripts/Systems/AdventureManager/AdventureManager.Calculations.cs · AdventureManager.Sequence.cs ·

Scripts/UI/Components/PreparationEventRateTooltip.cs · 기록: Docs/오늘도장비대여/Balance/Changelog/2026-07-29_보스_난이도계수_적용지점.md

3 — 개발 내용

3-2. 저장 시스템 — 반복되던 데이터 손실

경영 게임의 저장은 단순히 "진행을 보관"하는 게 아니라 게임 규칙의 일부입니다. 강제 종료로 결과를 되돌릴 수 있으면 강화·상자 같은 도박 요소가 무의미해지고, 파일이 한 번 깨지면 수십 시간의 회차가 사라집니다.

이 시스템이 하는 일

두 파일을 씁니다. 회차 데이터 `gamedata.json` (게임오버 시 삭제)과 영구 데이터 `legacy.json` (유산·설정·도감). 런타임 객체는 `ScriptableObject`를 참조로 들고 있다가 저장할 때만 ID 문자열로 굽고, 불러올 때 도메인 매니저가 ID로 다시 찾아 연결합니다.

문제 — 데이터 손실 버그가 세 가지 경로로 반복됐다

1. 쓰는 도중 종료되면 파일이 깨졌습니다. `File.WriteAllText` 가 반쯤 쓴 상태에서 앱이 죽으면 다음 실행에서 JSON 파싱이 실패하고, 코드는 "저장이 없다"로 판단해 새 게임을 시작했습니다.
2. 강제 종료 세이브스킴. 강화에 실패하면 앱을 죽여 되돌리는 것이 가능했습니다. 저장 시점이 "주기적"이었기 때문입니다.
3. 참조 복원 실패가 상태를 오염시켰습니다. 삭제된 던전 ID를 참조하는 진행 중 모험이 로드에서 드롭되면, 그 모험에 묶인 무기는 영원히 "대여 중", 모험가는 영원히 "모험 중"으로 남았습니다.

해결 — 규약 세 가지를 공통 장치로 고정

규약	구현
원자적 쓰기	모든 저장은 <code>SaveManager.WriteAllTextAtomic</code> 만 거칩니다. 임시 파일에 쓰고 <code>File.Replace</code> 로 교체하며 직전본을 <code>.bak</code> 으로 남깁니다. 로드 시 본 파일이 깨지면 <code>.bak</code> 으로 복구하고 손상본은 <code>.corrupt-시각</code> 으로 보존합니다. 저장 API는 성공 여부를 <code>bool</code> 로 돌려주고, 실패하면 마지막 저장 시각을 되돌리고 클라우드 업로드를 보류합니다.
확정 직후 저장	<code>SaveAfterCommittedAction(reason)</code> — 강화·상자·투자·모험 출발처럼 결과가 확정되는 지점에서 즉시 저장합니다. 이제 강제 종료해도 결과는 이미 파일에 있습니다.
로드 후 복구 패스	<code>RepairLoadedGameData()</code> — 복원에 실패해 드롭된 모험이 있으면 그 무기의 대여 상태와 모험가의 모험 중 상태를 풀고, 주인 없는 배정 아이템을 인벤토리로 돌려보냅니다.

```
// SaveManager.cs - WriteAllTextAtomic (요지): 임시 파일에 쓴 뒤 교체, 직전본은 .bak
File.WriteAllText(tmpPath, json);
File.Replace(tmpPath, path, bakPath);
```

영구 데이터는 한 단계 더 조심했습니다. 로드 함수가 `Success / Missing / RestoredFromBackup / Corrupted` 상태를 함께 돌려주고, **Corrupted면 빈 데이터로 조용히 시작하지 않고** 클라우드 복구나 사용자 안내를 거치게 했습니다. 유산은 잃으면 되돌릴 수 없기 때문입니다.

결과 — 알파 3회와 정식 출시 이후 세이브 손상·되돌리기 관련 제보가 없었습니다. 새 상태를 추가할 때 "어느 규약에 걸리는지 먼저 확인한다"를 체크리스트에 넣었습니다.

코드: `Scripts/Systems/SaveManager.cs` · `Scripts/Core/GameManager.cs` (`SaveAfterCommittedAction`, `RepairLoadedGameData`) · `Scripts/Utils/SaveDataConverter.cs`

3 — 개발 내용

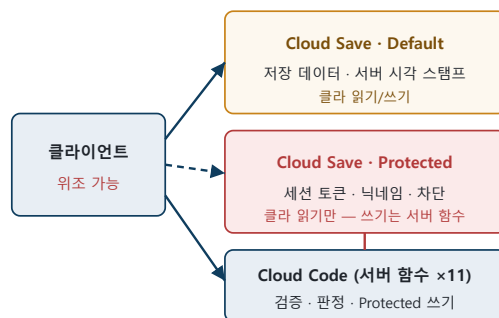
3-3. 온라인 — 리더보드 치팅과 세이브 동기화

리더보드·닉네임·기기 이전·계정 차단이 필요했지만 전용 서버를 운영할 여력은 없었습니다. Unity Gaming Services 위에서 "클라이언트가 쓰는 값은 전부 위조될 수 있다"는 전제로 설계했습니다.

이 시스템이 하는 일

익명 로그인 후 회차 데이터와 영구 데이터를 Cloud Save에 올리고, 리더보드에는 생존 일수를 제출합니다. 닉네임 설정, 24시간짜리 기기 이전 코드, 운영자의 계정 차단, 최소 버전 미만 차단이 있습니다.

핵심은 Cloud Save의 **Protected 영역**입니다. 클라이언트는 읽을 수만 있고 쓰기는 서버 함수(Cloud Code)만 할 수 있어서, "클라이언트가 절대 못 바꾸는 값"을 여기에 둡니다 — 세션 토큰, 닉네임, 차단 상태. 판정이 필요한 쓰기는 전부 Cloud Code 함수 11개로 옮겼고, 이 함수들은 git에 두고 GitHub Actions가 배포합니다.



문제 ① — 리더보드 점수를 클라이언트가 그냥 보낼 수 있었다

초기 구현은 클라이언트가 "N일 생존"을 그대로 제출했습니다. 값을 바꿔 보내거나, 한 번 받은 응답을 다시 보내거나, 게임을 시작하자마자 큰 값을 보내는 것이 전부 가능했습니다.

해결 — 서버 함수에서 4단계로 검증

1. **타입·범위** — 양의 정수, 상한 이하
2. **차단 상태** — Protected의 차단 키 재확인
3. **세션 토큰 대조 + 재사용 차단** — 게임 시작 시 `startGameSession` 이 서버에서 토큰을 만들어 Protected에 저장합니다. 제출 시 대조하고 사용된 플래그를 세웁니다. 이 플래그 갱신은 읽을 때 받아둔 `writelock`으로 조건부 갱신해서, 같은 토큰을 동시에 두 번 제출하면 한쪽만 성공합니다.
4. **시간 검증** — 세션 시작부터의 경과 시간이 주장한 일수에 비해 너무 짧으면 거부

검증을 하나 뺀 결정. 제출한 일수와 클라우드 저장본의 현재 일수를 대조해 거부하는 로직이 있었는데, 로 그만 남기도록 바꿨습니다. 둘 다 클라이언트가 쓰는 값이라 치터는 저장본을 먼저 위조하면 통과하는 반면, 정상 유저는 일자 경계에서 업로드가 한 번만 늦어도 거부당해 기록을 영구히 잃었습니다. 막지 못하는 치터보다 잃는 정상 유저가 더 큰 손해라고 판단했습니다.

문제 ② — 두 기기의 저장이 충돌하면 어느 쪽이 최신인지 알 수 없었다

클라이언트 시계로 "최신"을 정하면 기기 시간을 바꾸는 것만으로 오래된 저장이 이깁니다. 게임오버 뒤 메인메뉴에서 동기화하면 클라우드에 남은 죽은 회차가 되살아나는 문제도 있었습니다.

해결 — 시계는 서버 것만 믿고, 정리 순서를 고정

- 업로드 함수가 저장과 동시에 **서버 시각 스탬프**를 기록합니다. 클라이언트는 자기가 마지막으로 올린 스탬프만 기억합니다.
- 클라우드 스탬프가 내 마지막 업로드보다 크면 클라우드 채택, 같으면(= 내가 올린 본) 로컬 채택(오프라인 진행 가능성). 영구 데이터가 손상돼 있으면 스탬프를 무시하고 무조건 클라우드.
- 영구 데이터는 회차 데이터와 **독립적으로** 동기화 — 게임오버 직후 새 기기처럼 회차 데이터가 없어도 유산은 내려받아야 하므로.
- 게임오버 시 클라우드 회차 데이터를 비웁니다. 실패하면 플래그를 남겨 재시도하고, 이 정리는 **항상 다운로드보다 먼저** 실행되도록 호출 순서를 고정했습니다.
- UGS 초기화 10초 + 절차 10초의 이중 타임아웃. 상한이 지난 뒤 늦게 온 응답은 **버립니다** — 사용자가 이미 게임에 들어갔을 수 있어 로컬을 덮어쓰면 안 되기 때문입니다.

```
// CloudSyncService.cs - ResolveGameDirection (요지)
if (cloudStampMs > lastUploadedStampMs) return Direction.UseCloud; // 다른 기기가 더 최근에 올림
return Direction.UseLocal; // 내가 올린 본 = 로컬이 정본
```

문제 ③ — 기기를 바꾸면 익명 계정을 잃었다

익명 로그인은 기기에 묶입니다. 새 폰으로 옮기면 계정이 사라지므로 이전 수단이 필요했고, 동시에 남의 PlayerId만 알아서 남의 계정을 가져가는 일은 막아야 했습니다.

해결 — 24시간 1회용 복구 코드

서버가 "playerId-secret" 코드를 만들고 secret은 소유자의 Cloud Save에만 둡니다. 새 기기가 코드를 넣으면 서버가 secret을 대조해 데이터를 복사하고 사용 처리합니다. **secret 불일치와 데이터 없음**을 같은 **에러코드**로 응답해 "이 계정이 존재하는가"를 알아내는 용도로 쓸 수 없게 했습니다. 원래 기기는 30초 폴링으로 사용을 감지해 로컬·클라우드·복원 키를 모두 정리합니다.

결과

알파 0.0.3에서 "순위가 올라가지 않는다"는 제보 한 건이 있었고(SDK 에러코드 문제, 4장), 그 외 리더보드·동기화 관련 제보 없이 정식 출시했습니다. 운영자용 차단 관리 웹(FastAPI, Google 로그인)도 만들어 Unity 대시보드 없이 처리할 수 있게 했습니다.

코드: Tools/CloudCode/submitLeaderboardScore.js · redeemRestoreCode.js · Scripts/Systems/CloudSyncService.cs · Scripts/Systems/UGSManager.cs · Tools/admin-site/

3 — 개발 내용

3-4. 밸런싱 — 근거 없는 수치 조정

경영 게임은 수치가 곧 재미입니다. 무기 가격, 벌금, 의뢰 난이도, 유산 효과 — 수십 개의 값이 서로 얽혀 있어서 하나를 고치면 다른 데가 무너졌습니다.

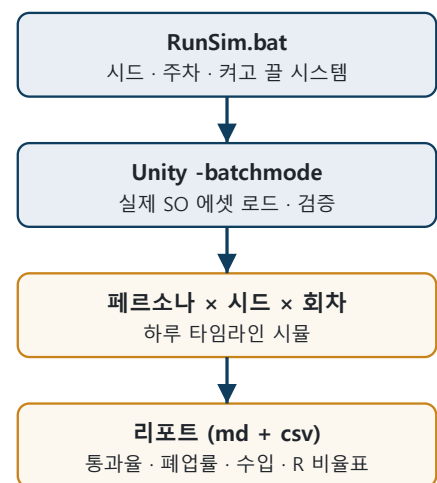
문제 — "무기 3,000G가 비싼지 싼지 판단할 근거가 없었다"

초기 밸런싱은 직접 플레이해 보고 감으로 고치는 방식이었습니다. 한 회차가 40주(실시간 5시간 이상)라 한 번 바꾸고 확인하는 데 하루가 걸렸고, 목표를 "초급자가 15주쯤 버티게"처럼 결과값에 걸어 두니 A를 고치면 B가 깨지는 두더지 잡기가 됐습니다. 두 가지가 없었습니다 — **빠르게 결과를 볼 수단과 가격을 읽는 기준**.

해결 ① — 게임 로직을 헤드리스로 돌리는 밸런스 시뮬레이터

Unity 에디터 확장으로 시뮬레이터를 만들었습니다. 실제 Config·던전·무기·의뢰 ScriptableObject를 그대로 읽고(수치를 바꾸면 시뮬 코드는 손댈 필요 없음), 계산식은 런타임 코드를 미러링합니다. 플레이어 대신 **페르소나 3종**(상급/중급/초급 — 차이는 정보를 얼마나 사는가 하나뿐)이 하루를 AM 6 ~ PM 9 타임라인으로 돌며 방문·배정·의뢰·야침 이벤트를 처리합니다.

- 배치 파일 → Unity batchmode → 100시드 × 100주 × 3페르소나 × 최대 5회차(폐업 시 유산 획득 후 재시작)를 몇 분에 돌리고 마크다운 리포트를 씁니다.
- 축별로 난수 스트림을 분리해(메인/특성/점술/네임드/의뢰) 새 시스템을 끈 실행이 도입 이전과 **비트 단위로 같은 결과**를 내게 했습니다. "새 시스템이 결과를 바꿨는가"를 이걸로 검증합니다.
- 같은 시드끼리 짝지은 차이의 중앙값을 써서 시드 경로 발산 노이즈($\pm 10\%$)에 효과가 묻히지 않게 했습니다.



해결 ② — 모든 가격을 "모험 1회 순수입(R)"의 배수로 읽는 기준

이 게임의 골드는 전부 모험 성공에서 나옵니다. 그래서 **R = 모험 1회 성공 시 순수입**을 기준으로 잡고, 무기·재료·대장간·수색·점술 가격을 "모험 몇 회분"으로 정의했습니다(무기 1.8R~30R, 수색 0.3R, 점술 0.6R). 조정 순서도 고정했습니다 — R을 먼저 확정하고, 가격을 R 배수로 맞추고, 벌금을 수입 비율로 걸고, **생존 주차는 조정 대상이 아니라 검증 대상**으로 봅니다. 시뮬레이터가 이 비율표를 구간별로 출력합니다.

결과 ① — "쉬움" 의뢰가 "최고난도"보다 어려웠던 것을 발견하고 고쳤다

40주 캠페인 이후의 엔드리스 의뢰 16종은 기획 단계에서 휴식형/표준형/시험형으로 나눠 두었습니다. 시뮬레이터로 템플릿 하나씩 단독 반복시켜 보니 실측 통과율은 그 분류와 무관했고 심지어 역전돼 있었습니다.

템플릿 (기획 분류)	실측 통과율
W70 시험형 — 최고난도	96.7%
W61 휴식형 — 쉬움	36.8%

난이도를 만드는 것은 아키타입이 아니라 **병목 3종**(저가용 던전 요구 / 희소 무기 요구 / 돌의 조합)이었습니다. 등급 요구는 병목이 아니었습니다 — 전설 이상 2회 단독 요구가 90.7%.

수정 — 난이도 등급 4단계

(Easy/Normal/Hard/Extreme)를 **병목 개수 기준**으로 새로 정의하고, 61주 이후 별금을 고정값에서 주차 비례 곡선으로 바꿨습니다.

별금 기울기 A/B — 기울기 4,000에서는 상급자 폐업률 31% vs 중급자 21%로 **순서가 역전**됐습니다 (재투자로 현금이 얇은 상급자가 먼저 죽음). 8,000에서 정상화. 직관으로는 4,000을 골랐을 겁니다.

수정 후 스왑 (상급자)	통과율	폐업률	주간 순수입
수정 전 — 16종 전부 "Easy"	44~46%	65~66%	-76,000G
W41 [Easy]	97.6%	0.0%	+104,853G
W45 [Normal]	94.6%	0.0%	+93,683G
W49 [Hard]	74.6%	29.4%	+23,724G
W53 [Extreme]	46.7%	60.5%	-57,502G

라벨 순서와 통과율 순서가 일치하게 됐습니다.

결과 ② — 목표 통과율 달성

레벨디자인 문서의 합격선은 상급 95% 이상 / 중급 70% / 초급 40% 미만이었습니다. 정식 출시 직전 실측은 **96% / 70% / 33%**로 세 층 모두 목표 안에 들어왔습니다. 이후 모든 수치 조정은 "근거 → 대상 → 전후 → 실측"을 적은 기록을 남기고 있습니다.

한계

- 계산식을 미러링하므로 런타임 공식이 바뀌면 시뮬레이터도 손으로 맞춰야 합니다. 캠페인 길이를 상수로 뒀다가 조용히 어긋난 적이 있어 Config에서 읽도록 고쳤습니다.
- 봇 정책이 결과를 지배합니다. 서열은 믿고 절대값은 참고만 합니다. "지표가 0이면 밸런스 결론 전에 봇부터 의심하라"는 체크리스트를 남겼습니다.

코드: Scripts/Editor/BalanceSimulator/ · Tools/BalanceSim/ · 기록: Docs/오늘도장비대여/Balance/Reference/밸런스_시뮬레이터_정리.md · Balance/Changelog/2026-07-29_엔드리스_난이도등급과_별금곡선.md

3 — 개발 내용

3-5. 다국어 — 3개 언어 추가

알파 0.0.2에서 영어·일본어·중국어(간체)를 추가했습니다. 번역 자체보다 클라이언트 쪽 문제 두 가지가 컸습니다 — 한글 전용 폰트로 만들어진 UI에서 일본어·중국어를 어떻게 그릴 것인가, 그리고 수천 개 문자열 중 빠진 것이 없다는 것을 어떻게 보장할 것인가.

문제 ① — 기존 UI가 한글 전용 폰트에 묶여 있어 일본어·중국어가 표시되지 않았다

전 UI가 한글 전용 폰트 하나를 82개 파일 1,100여 곳에서 참조하고 있었습니다. 가나·한자 글리프가 없어 일본어·중국어는 □로만 나왔습니다. 정공법은 로케일별로 폰트를 교체하는 것이지만, 텍스트마다 붙은 머티리얼 프리셋(외곽선·그림자)까지 흔들려 수백 개 프리팹을 다시 만져야 했습니다.

해결 — 폰트는 그대로 두고 풀백 순서만 바꾼다

배정 폰트를 바꾸지 않고 일본어·중국어 폰트를 풀백 폰트로 추가했습니다. 로케일이 바뀌면 풀백 순서만 재배열합니다(`ja` → [`JP`, `SC`], `zh` → [`SC`, `JP`]). 순서가 중요한 이유는 일본어와 중국어가 같은 코드포인트를 쓰는 한자가 풀백 1순위 폰트의 자형으로 그려지기 때문입니다(Han unification). 프리팹은 한 개도 건드리지 않았습니다. 아틀라스는 글리프 수를 실측해 결정했습니다 — 한글은 11,172자로 닫혀 있어 전부 미리 굽고, 닉네임처럼 열거할 수 없는 CJK 입력은 런타임 동적 아틀라스에 맡겼습니다. 상용 한자 합집합을 굽는 안은 용량이 4배가 되고도 인명 한자를 못 덮어 기각했습니다.

문제 ② — 문자열 약 2,300개 중 "빠진 게 없다"를 증명할 수 없었다

코드·프리팹·데이터에 흩어진 한국어를 전부 String Table로 옮겨야 했는데, 완료 판정을 "프리팹의 한글 수 - 로컬라이즈 컴포넌트 수"로 하고 있었습니다.

해결 — 스크립트로 검사한다

이 방법이 두 번 틀렸습니다. 한글을 세는 정규식이 유니코드 D 대역(평·표 등)을 통째로 놓쳤고, 텍스트 필드 밖의 한국어(사운드 표시명)를 보지 않았습니다. 그래서 PowerShell 감사 스크립트를 만들었습니다 — 프리팹·씬 YAML을 훑어 텍스트 컴포넌트마다 어느 스크립트의 어느 필드가 값을 쓰는지 역추적해 6종(제외 / 부착됨 / 코드가 쓰므로 부착 금지 / 사람 확인 / 스크립트 불명 / 누락)으로 판정하고, 검증 스크립트가 4개 로케일 키 일치·빈 번역 0·영·일·중 테이블에 남은 한국어 0 등 9항목을 종료 코드로 검사합니다.

결과

프리팹 수정 없이 알파 0.0.2에 3개 언어를 실었고, 검증 9항목을 통과한 상태로 정식 출시했습니다.

코드: `Scripts/Systems/LocalizationManager.cs` · `Scripts/Systems/DataLocalizer.cs` · `Tools/Localization/` · 기록: `Docs/오늘도장비대여/Development/다국어_도입전략.md`

3 — 개발 내용

3-6. 기획 데이터 툴 — 조용히 깨지는 데이터

무기 178종, 던전 28종, 의뢰 56종 같은 기획 데이터는 ScriptableObject로 관리하지만, 대량 편집은 스프레드시트가 편합니다. SO ↔ CSV 왕복 툴을 만들었고, 그 과정에서 가장 무서운 버그를 만났습니다.

문제 — 컬럼이 밀리면 빌드는 통과하고 데이터만 깨진다

SO에 필드를 하나 추가하고 CSV 임пор터를 안 고치면, 이후 컬럼 값이 전부 한 칸씩 밀려 엉뚱한 필드에 들어갑니다. 타입이 우연히 맞으면 오류도 없습니다. 무기 가격이 무기 등급 자리에 들어간 채 며칠을 모르고 지나간 적이 있습니다.

해결 — 쓰기 전에 diff를 보여 주고, 스키마를 선언해 검증한다

탭	동작
Export	SO → 미리보기 CSV만 생성하고 원본 CSV와 diff를 표시. 원본은 건드리지 않음
Applier	미리보기 → 원본 덮어쓰기를 별도 승인 단계로 분리
Import	CSV → SO 변환 결과를 메모리에서 먼저 계산해 diff를 띄우고, 체크한 항목만 의존 순서대로 적용

임포트는 타입마다 스키마를 규칙 배열로 선언하고, 컬럼 수·타입·enum 정의 여부를 먼저 검사합니다. 오류가 한 건이라도 있으면 그 타입 전체를 중단해 반쯤 적용된 상태를 만들지 않습니다.

```
// CSVImportTab.cs - Apply_WeaponData
if (Validate("WeaponData", 9,
    new[] { "str", "str", "str", "enum:Grade", "enum:WeaponType", "int", "str", "int", "str" }) > 0)
    return; // "WeaponData 12행: 컬럼 부족 (8/9)" / "grade: 'Rara'는 Grade가 아님" ...
```

diff는 StaticID를 키로 양쪽을 합쳐 추가/삭제/변경을 판정하고, 변경은 `basePrice: 100 → 120` 처럼 컬럼 단위로 보여 줍니다. 개행·공백 차이로 가짜 diff가 뜨지 않게 정규화하고, 엑셀에 열린 파일은 잠금을 감지해 경고합니다.

결과

이후 데이터 밀림 사고는 없었습니다. "SO 필드를 건드리면 같은 작업에서 Export-Import-검증 규칙을 함께 고치고 diff를 한 번 돌린다"를 규칙으로 문서에 고정했습니다. 같은 방식으로 밸런스 시뮬레이터(3-4), 디버그 대시보드, 모험가·NPC 외형 미리보기, 의뢰 자동 생성기 등 에디터 툴을 만들어 썼습니다.

코드: Scripts/Editor/CSVImportTab.cs · CSVDiffCore.cs · CSVExportTab.cs

트러블슈팅

3장에 넣기엔 작지만, 원인을 찾는 과정이 기억에 남는 버그들입니다.

① 순위가 올라가지 않는다 — SDK가 에러코드를 뭉갸다

현상 알파 0.0.3 제보. 차단된 계정이 아닌데 리더보드 제출이 조용히 실패. **원인** 서버 함수는 차단 시 에러코드 2006을 던지는데, Unity Leaderboards SDK가 특정 경로에서 이를 9009(일반 오류)로 바꿔 전달했습니다. 클라이언트는 9009를 "재시도 불가"로 보고 보류 점수를 폐기했습니다. **해결** 에러 메시지 문자열 안에 서버가 넣은 JSON(`{"reason": "...", "until": "..."}`)이 남아 있는 것을 확인하고, 정규식으로 추출해 이중 분기했습니다. 정상 실패는 재시도 큐로, 차단은 안내 팝업으로.

② 신비 상자가 일반 상자와 같은 확률로 나왔다 — 디버그 값이 출시본에 남았다

현상 시뮬레이터에서 중급자 상자 지출이 예상보다 컸습니다. **원인** 상자 등급 가중치 Config가 코드 기본값 0.60/0.30/0.10이 아닌 0.33/0.33/0.33. 이력을 추적하니 6월의 "morningevent debug" 작업에서 테스트용으로 균등화한 뒤 되돌리지 않은 것이었습니다. 런타임은 에셋을 읽으므로 10,000G 상자가 1,500G 상자와 같은 확률로 등장하고 있었습니다. **해결** 값 복원 + 시뮬레이터로 전후 확인(중급자 상자 평균 가격 5,159G → 3,436G). 이후 디버그용 값 변경은 Config가 아닌 에디터 창의 오버라이드로만 하도록 바꿨습니다.

③ 한글 4건이 번역에서 빠졌다 — 정규식이 D 대역을 놓쳤다

현상 영어 화면에 한국어 4건 잔존. **원인** 프리팹 YAML에서 한글을 세는 정규식이 `\u[A-C][0-9A-F]{3}` 였는데 한글 완성형은 AC00~D7A3. 첫 글자가 평·표처럼 D 대역이면 그 줄 전체가 집계에서 빠졌습니다. **해결** 범위 수정 + 집계 방식 자체를 "텍스트 컴포넌트 하나하나를 역추적"하는 방식으로 바꿨습니다(3-5).

④ 배치 파일이 종료 코드 0으로 아무것도 안 했다 — 코드페이지

현상 시뮬레이터 배치 실행이 성공으로 끝났는데 리포트가 없음. **원인** .bat 에 넣은 한글 주석을 cmd.exe 가 OEM 코드페이지로 읽어 파싱이 깨졌고, 그 뒤 명령이 실행되지 않은 채 정상 종료. **해결** 배치 파일은 ASCII만 쓰고, 이 함정을 모든 배치 파일 헤더에 적어 두었습니다.

⑤ 캠페인 길이를 줄였더니 시뮬레이터가 조용히 어긋났다

현상 캠페인을 60주에서 40주로 압축한 뒤 시뮬 결과의 엔드리스 구간이 이상함. **원인** 시뮬레이터가 캠페인 길이를 상수로 들고 있었습니다. **해결** Config에서 읽도록 수정. "수치는 전부 에셋에서 읽는다"는 원칙의 예외가 하나 남아 있었던 것이라, 상수로 둔 값이 더 없는지 전수 확인했습니다.

출시와 운영

Google Play 알파 트랙에서 3회 테스트를 거쳐 정식 출시했습니다. 테스트 지적은 대부분 "문구"가 아니라 "이해"와 "리듬"에 관한 것이었고, 대응도 UI 요소가 아니라 시스템 확장이었습니다.

일정

시기	버전	내용
2025.12	—	개발 시작
2026.07.30	0.0.1 알파	대장간 강화 재료를 진화 재료로 개편, 성공률 상세 패널, 새 레시피
2026.08.03	0.0.2	앱 이름 한국어화, 영어·일본어·중국어(간체) 추가, 대화창 가시성
2026.08.04	0.0.3	저녁 스킵, 순위 업로드 버그, 본인 닉네임 표시. 같은 날 강제 업데이트·계정 삭제·서버 함수 자동 배포 정비
2026.08.17	1.0.0 정식	Google Play 정식 출시

플레이테스트 지적과 대응

지적	대응
"성공률 텍스트가 뭘 의미하는지 모르겠다"	문구 수정이 아니라, 누르면 요소별 기여도를 펼쳐 보이는 상세 패널 신설 (3-1)
"저녁에 할 일이 없어서 맥이 끓긴다"	아침에 있던 "상호작용 다 끝나면 스킵" 패턴을 저녁으로 대칭 확장 — 기존 <code>SkipMorning()</code> 을 재사용해 저비용으로 해결
"앱 이름이 영어다"	한국어화하면서 앱 이름도 String Table로 옮김 → 이후 일본어·중국어 앱 이름이 얹히는 자리가 됨
"대화창의 이름/역할이 잘 안 보인다"	표시 계층·대비 수정

운영을 위해 만든 것

- **서버 함수 자동 배포** — Cloud Code 함수는 git이 단일 원본. `main` 에 올리면 GitHub Actions가 diff → 배포 → diff로 반영하고, 대시보드에서 손으로 고친 흔적은 다음 배포에서 되돌립니다.
- **애널리틱스 스키마 검사** — UGS Analytics는 미등록 이벤트를 조용히 버립니다. 이벤트 정의 파일과 C# 호출부를 CI가 대조해 어긋나면 실패시킵니다.
- **운영자 웹** — 계정 차단·해제와 리더보드 정리를 Google 로그인 + 이메일 화이트리스트로 처리. DB 없이 Cloud Save를 진실의 원천으로 두어, 이 웹이 죽어도 Unity 대시보드에서 같은 키를 조작할 수 있게 했습니다.
- **강제 업데이트** — 최소 버전 미만은 진입 차단. 판정을 클라이언트가 하고 결과를 캐시해 오프라인에서도 같은 동작을 재현합니다.

회고

혼자 끝까지 만들어 본 덕에, 잘한 것보다 부족한 것이 더 또렷하게 보입니다.

배운 것

- **측정할 수 없는 것은 조정할 수 없다.** 시뮬레이터를 만들기 전의 밸런싱은 매번 임의 수치였습니다. 도구가 생기자 "쉬움 의뢰가 최고난도보다 어렵다" 같은, 플레이해서는 절대 못 찾았을 문제가 드러났습니다.
- **클라이언트가 쓰는 값은 판정에 쓰지 않는다.** 서버 시각 스탬프, 서버만 쓰는 저장 영역, 조건부 갱신 — 이 세 가지로 위조와 경합 대부분이 정리됐습니다. 반대로 "정상 유저가 잃는 검증"은 빼는 게 맞다는 것도 배웠습니다.
- **실측이 계획을 뒤집으면 계획을 버린다.** 다국어 아틀라스는 처음 계획대로 갔으면 4배 무거워지고도 목적을 못 이뤘을 겁니다.
- **안전망은 사고를 맞고 자란다.** 저장 규약 3가지, 감사 스크립트의 판정 항목, 배치 파일 주의 사항 — 전부 실제로 한 번 당한 뒤에 생겼습니다. 그 경위를 문서에 남겨 같은 사고를 두 번 맞지 않게 했습니다.

부족한 것

- **자동 테스트가 없습니다.** 에디터 툴의 diff-검증, 검사 스크립트, CI 대조로 안전망을 깔았지만 게임 로직의 회귀 테스트는 없습니다. 이 부재가 "거대 매니저의 내부 모듈 분리"는 회귀 위험이 커서 보류한다"는 결정을 낳았습니다.
- **시뮬레이터가 계산식을 이중 관리합니다.** 수치는 에셋에서 읽지만 공식은 미러라, 런타임 공식이 바뀌면 손으로 맞춰야 합니다.
- 일부 파일이 지나치게 큼니다(튜토리얼 매니저 약 2,900줄). 다른 매니저에 적용한 partial 분할 규칙을 여기엔 적용하지 못했습니다.

다음에 다시 한다면

도메인 매니저의 계산 로직을 처음부터 MonoBehaviour 없는 순수 C#으로 분리하겠습니다. 그러면 EditMode 테스트와 밸런스 시뮬레이터가 **같은 코드**를 쓰게 되어, 위의 "테스트 부재"와 "이중 관리"가 한번에 해결됩니다. 두 문제가 같은 뿌리에서 나왔다는 것이 이 프로젝트에서 얻은 가장 큰 교훈입니다.

저장소 안내 — GitHub 저장소는 유료 에셋과 아트·사운드를 제외한 자체 작성 코드와 문서만 담은 열람용입니다. 본문의 코드 경로는 저장소의 같은 경로를 가리킵니다. 설계 문서와 밸런스 변경 기록, 시뮬레이션 리포트 샘플이 Docs/에 있습니다.

github.com/seunghee-2002/Todays-Weapon-Rental-code · [Google Play](#) · [소개 페이지](#)